

Compiladores — Análise Léxica

Análise Léxica com ANTLR 3 — Subconjunto de C

14.06.2026

Jéssica Pereira, Kauã Bispo, Kenandja Krishna, Liriel Gomes, Lucas Ramos

1	Introdução	3
2	O Gerador ANTLR	8
3	A Gramática	11
4	Análise Léxica na Prática	17
5	Conclusão	22

1 Introdução

Sobre este Trabalho

4 / 24

Disciplina: Compiladores

Professor: Alexandre Paes

Instituição: Universidade Federal de Alagoas — Campus Arapiraca

Equipe:

1. Jéssica Pereira
2. Kauã Bispo
3. Kenandja Krishna
4. Liriel Gomes
5. Lucas Ramos

Gerador utilizado: ANTLR 3

Linguagem alvo: Java

Gramática: `CLexer.g` — Lexer para um subconjunto de C (Soma dos Pares)

“A análise léxica, também conhecida como *scanner* ou leitura, é a **primeira fase** de um processo de compilação e sua função é fazer a leitura do programa fonte, caractere a caractere, agrupar os caracteres em **lexemas** e produzir uma sequência de símbolos léxicos conhecidos como **tokens**.”

— Johny Douglas — *Compiladores para Humanos*

O que é Análise Léxica? (ii)

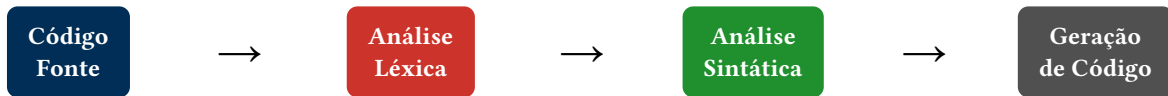
6 / 24

Na prática, dado o texto de entrada:

```
#include <stdio.h>

int main() {
    int soma = 0;
    for (int i = 0; i <= 10; i++) {
        if (i % 2 == 0) {
            soma = soma + i;
        }
    }
    printf("%d", soma);
    return 0;
}
```

O analisador léxico reconhece cada pedaço significativo e o classifica como um **token**.



A **análise léxica** é a **primeira etapa**: transforma o texto bruto em uma sequência de tokens que o parser irá consumir.

2 O Gerador ANTLR

O que é o ANTLR?

ANTLR — *Another Tool for Language Recognition*

Criado por **Terence Parr** (Universidade de São Francisco)

- Ferramenta de geração de **Lexers** e **Parsers** a partir de uma gramática **.g**
- Amplamente utilizado na construção de linguagens de programação, frameworks e ferramentas de análise
- Suporta múltiplas linguagens alvo: Java, Python, C, C#, C++, PHP

Neste trabalho, geramos o código em **Java**.

Você escreve:

- Um arquivo `NomeDaGramatica.g`
- Regras do **Lexer** (maiúsculas)
- Regras do **Parser** (minúsculas)
- Opção de linguagem alvo

Executa:

```
antlr3 CLexer.g
```

O ANTLR gera automaticamente:

- `CLexer.java` — lexer gerado
- `CLexer.tokens` — tabela de tokens
- Compilar com
`javac -cp antlr-3.x.jar CLexer.java`

O `CLexer.java` é a implementação do lexer que realiza a tokenização da entrada em Java.

3 A Gramática

Código fonte analisado:

```
#include <stdio.h>

int main() {
    int soma = 0;
    for (int i = 0; i <= 10; i++) {
        if (i % 2 == 0) {
            soma = soma + i;
        }
    }
    printf("%d", soma);
    return 0;
}
```

T (terminais): INT, FOR, IF, RETURN, INCLUDE, STRING, ID, NUM, EQ, LE, INC, PLUS, ASSIGN, MOD, DOT, LPAREN, RPAREN, LBRACE, RBRACE, SEMI, LT, GT, COMMA

Este projeto usa apenas lexer

O arquivo atual não possui parser. Ele define somente o lexer `CLexer.g`, que reconhece palavras-chave, identificadores, números, operadores, strings, delimitadores e comentários.

Regras do Lexer

```
INT      : 'int';
FOR      : 'for';
IF       : 'if';
RETURN   : 'return';
INCLUDE  : '#include';
STRING   : '"' (~'"')* '"';
ID       : ('a'..'z'|'A'..'Z'|'_'|
('a'..'z'|'A'..'Z'|'0'..'9'|'_'|
NUM      : '0'..'9'+;
EQ       : '==';
LE       : '<=';
INC      : '++';
PLUS     : '+';
ASSIGN   : '=';
```

```
MOD      : '%';  
DOT      : '.';  
LPAREN   : '(';  
RPAREN   : ')';  
LBRACE   : '{';  
RBRACE   : '}';  
SEMI     : ';';  
LT       : '<';  
GT       : '>';  
COMMA    : ',';  
WS       : (' '|'\t'|\r'|\n')+  
{ skip(); };  
COMMENT  : '//' (~'\n')* { skip(); };
```

`skip()`: espaços e comentários são descartados antes de gerar os tokens.

Tabela de Tokens Gerada

15 / 24

Após executar o ANTLR, o arquivo `CLexer.tokens` contém:

Token	Tipo	Descrição
<code>INT</code> , <code>FOR</code> , <code>IF</code> , <code>RETURN</code> , <code>INCLUDE</code>	Literal	Palavras-chave individuais
<code>STRING</code>	Regular	<code>'" ' (~'" ')* '" ' — strings entre aspas (ex.: <code>"%d"</code>)</code>
<code>ID</code>	Regular	<code>('a'..'z' 'A'..'Z' '_')</code> <code>('a'..'z' 'A'..'Z' '0'..'9' '_')*</code>
<code>NUM</code>	Regular	<code>'0'..'9' +</code>
<code>EQ</code> , <code>LE</code> , <code>INC</code>	Literal	Operadores multi-caractere: <code>==</code> , <code><=</code> , <code>++</code>
<code>PLUS</code> , <code>ASSIGN</code> , <code>MOD</code> , <code>DOT</code>	Literal	Operadores: <code>+</code> , <code>=</code> , <code>%</code> , <code>.</code>

Tabela de Tokens Gerada (ii)

16 / 24

Token	Tipo	Descrição
LPAREN, RPAREN, LBRACE, RBRACE, SEMI, LT, GT, COMMA	Literal	Delimitadores e pontuação
WS	Regular	Espaços/quebras — descartados via <code>skip()</code>
COMMENT	Regular	Comentários de linha <code>//</code> — descartados via <code>skip()</code>
EOF	Implícito	Fim da entrada

O ANTLR classifica tokens em: **literais**, **regulares** e **implícitos** (EOF).

4 Análise Léxica na Prática

O Arquivo de Gramática (.g)

O arquivo `CLexer.g` é o **ponto de partida**:

- Extensão `.g` = gramática **ANTLR 3**
- Nome do arquivo = nome declarado em `lexer grammar`
- Opção `language = Java` define a linguagem alvo
- Apenas regras do **lexer** (MAIÚSCULAS)
- `WS` e `COMMENT` usam `skip()` (ANTLR 3)

Para gerar os arquivos Java:

```
antlr3 CLexer.g
javac -cp antlr-3.x.jar CLexer.java
TestLexer.java
java -cp .:antlr-3.x.jar TestLexer
```

```
lexer grammar CLexer;

options { language = Java; }

// Palavras-chave (antes de ID)
INT      : 'int';
FOR      : 'for';
IF       : 'if';
RETURN   : 'return';
INCLUDE  : '#include';

// Strings (para o printf)
STRING   : '"' (~'"')* '"';

// Identificadores e números
ID       : ('a'..'z'|'A'..'Z'|'_'|
           ('a'..'z'|'A'..'Z'|'0'..'9'|'_'|'_')*);
NUM      : '0'..'9'+;

// Operadores (multi-char antes de single-
char)
```

```
EQ  : '=='; LE : '<='; INC : '++';
PLUS : '+'; ASSIGN : '=';
MOD  : '%'; DOT   : '.';

// Delimitadores
LPAREN : '('; RPAREN : ')';
LBRACE : '{'; RBRACE : '}';
SEMI   : ';'; LT : '<'; GT : '>';
COMMA  : ',';

// Ignorados
WS      : (' '|'\t'|\r'|\n')+ { skip(); };
COMMENT : '//' (~'\n')*      { skip(); };
```

Passo	Componente	O que faz
1	Input Stream	<code>ANTLRStringStream(code)</code> — encapsula a string caractere a caractere
2	Lexer (AFD)	<code>CLexer(input)</code> — percorre a entrada e agrupa em lexemas
3	Token Stream	<code>CommonTokenStream(lexer)</code> — coleta os tokens em uma fila
4	Iteração	<code>lexer.nextToken()</code> — retorna o próximo token até EOF
5	Saída	Imprime linha, coluna, tipo e valor de cada token reconhecido

Entrada: `SomaPares.c` — programa C que soma os pares de **0 a 10**

Linha	Coluna	Tipo	Valor
1	1	INCLUDE	'#include'
1	9	LT	'<'
1	10	ID	'stdio'
1	15	DOT	'.'
1	16	ID	'h'
1	17	GT	'>'
3	1	INT	'int'
3	5	ID	'main'
3	9	LPAREN	'('
3	10	RPAREN	')'
3	12	LBRACE	'{'
4	5	INT	'int'
4	9	ID	'soma'

Linha	Coluna	Tipo	Valor
4	14	ASSIGN	'='
4	16	NUM	'0'
4	17	SEMI	';'
5	5	FOR	'for'
5	9	LPAREN	'('
5	10	INT	'int'
5	14	ID	'i'
5	16	ASSIGN	'='
5	18	NUM	'0'
5	19	SEMI	';'
...			
Total de tokens: 62			

5 Conclusão

Resumo

O que aprendemos:

- A **análise léxica** é a 1ª fase da compilação
- Transforma texto em uma sequência de **tokens**
- O Lexer é implementado como um **Autômato Finito Determinístico (AFD)**
- O token `WS` com `skip()` é descartado antes de chegar ao Parser

Ferramentas utilizadas:

- ANTLR 3
- Runtime Java do ANTLR3
- `javac` / `java` (JDK)

Tipos de tokens no programa analisado:

- **Palavras-chave** — `INT`, `FOR`, `IF`, `RETURN`, `INCLUDE`
- **Identificadores (ID)** — `main`, `soma`, `printf`, `i`
- **Literais numéricos (NUM)** — `0`, `2`, `10`
- **Strings (STRING)** — `"%d"`
- **Operadores** — `EQ (==)`, `LE (<=)`, `INC (++)`, `PLUS`, `ASSIGN`, `MOD`
- **Delimitadores** — `LPAREN`, `RPAREN`, `LBRACE`, `RBRACE`, `SEMI`, `LT`, `GT`, `COMMA`
- **WS** e **COMMENT** são descartados via `skip()`

- **DOUGLAS, Johnny.** *Compiladores para Humanos*. GitBook. Disponível em: johnidouglas.gitbook.io
- **ANTLR.** *Runtime C Documentation*. Disponível em: antlr.org
- **Repositório do projeto:** disponível na descrição do vídeo (GitHub)